ProofQA

Stratégie de test ProofQA v1

Une démarche de test fondée sur les risques, appliquée à un produit réel, du cadrage jusqu'à la décision de lancement.

Version de présentation · Octobre 2026

32	21	47	19	186
user stories	conditionnent le lancement	risques analysés	critères de sortie	scénarios d'acceptation

À propos de ce document

ProofQA est une plateforme SaaS de Quality Assurance : elle analyse le produit web d'une entreprise et lui restitue un score sur 100, avec des anomalies reproductibles, puis un expert QA peut l'accompagner.

Ce document présente la stratégie de test conçue pour sa première version. Il en montre la démarche et les choix structurants. La stratégie complète, utilisée par l'équipe, reste confidentielle.

Auteur	Ghislain TOUSSI KAMGA, QA Engineer et Test Lead, ISTQB Foundation v4
Produit	ProofQA, plateforme Managed SaaS de QA, éditée par Titimbē Forge
Référentiel	ISTQB Foundation v4 : test basé sur les risques, tests au plus tôt (shift-left)
Périmètre	Plateforme client, back-office interne et espace client de la version 1

01 · POINT DE DÉPART

Pour qui teste-t-on, et pourquoi

Une stratégie de test commence par l'utilisateur et son problème. Ses attentes deviennent les critères de qualité, ses parcours deviennent les scénarios de bout en bout.

Le client

Éditeurs, ESN, startups et fintechs de 50 développeurs au plus, sans équipe QA structurée.

L'utilisateur principal

Le CTO ou Tech Lead, qui interprète les résultats et organise les corrections.

Son problème

Il livre en production sans preuve que son logiciel est fiable, ni pour lui, ni pour ses clients, ni pour ses investisseurs.

Le contexte d'usage

Application web responsive, pensée d'abord pour le mobile ; réseau 3G ; français et anglais.

Ce que chaque partie prenante attend du test

Partie prenante	Ce qu'elle attend	Conséquence pour le test
CTO client	Un verdict qu'il comprend seul	Tests de compréhension avec des utilisateurs réels
Développeur qui reçoit un résultat partagé	Une preuve qu'il peut reproduire	Chaque anomalie affichée est rejouée sur un site de référence
Investisseur ou client du client	Un résultat qui ne promet pas plus qu'il ne vérifie	Mention de périmètre et vocabulaire contrôlés partout
Équipe interne	Des données fiables, des accès maîtrisés	Exactitude des données, contrôle d'accès, traçabilité
Tiers dont le site est analysé	Ne pas être analysé sans autorisation	Consentement obligatoire, analyse non intrusive

Une démarche en neuf étapes

Chaque étape s'appuie sur la précédente : on ne priorise pas sans objectif, on n'analyse pas les risques sans exigences, on ne fixe pas de critères de lancement sans risques.

1. **Décrire le produit** — Client, utilisateur, problème, objectif de la version
2. **Prioriser le périmètre** — Ce qui est livré, ce qui ne l'est pas, dans quel ordre
3. **Formuler les exigences** — User stories reliées aux écrans
4. **Analyser les risques** — Risques produit et projet, cotés et classés
5. **Définir l'approche** — Niveaux, types, techniques et outils
6. **Fixer les critères** — Entrée en test, fin de ticket, lancement
7. **Organiser le travail** — Rôles, environnements, données, anomalies
8. **Écrire l'acceptation** — Règles à vérifier et scénarios de chaque story
9. **Tracer ce qui reste** — Actions et décisions en attente

02 · PÉRIMÈTRE

Prioriser avant de tester

Le périmètre est classé avec la méthode MoSCoW, au regard de l'objectif de la version. Une seule question tranche : « si cette fonction manque le jour du lancement, est-ce que je refuse de lancer ? »

Niveau	Définition	Règle de test
Must	Sans elle, la version n'est pas lancée	Testée à fond ; un défaut bloquant ou majeur empêche le lancement
Should	Prévue, désactivée si elle n'est pas prête et testée	Testée dès sa livraison ; sinon, on vérifie qu'elle est invisible
Could	Si le temps le permet	Aucune en v1 : chaque cas a été tranché
Won't	Hors version	On vérifie son absence

Répartition des 32 user stories



25 fonctionnalités sont explicitement classées Won't (paiement en ligne, rapport PDF, historique, alertes…). Les écrire noir sur blanc évite deux erreurs : les tester inutilement, ou se voir reprocher de ne pas les avoir testées.

03 · EXIGENCES

Des user stories centrées sur l'utilisateur

Chaque besoin s'écrit : **En tant que** [qui], **je veux** [quoi], **afin de** [pourquoi]. Le « afin de » porte la valeur pour l'utilisateur, jamais pour l'entreprise : c'est elle que le test d'acceptation vérifie.

CLI-06 Must En tant que CTO, je veux consulter mon score et mes résultats détaillés afin de connaître l'état réel de mon produit et savoir quoi corriger en priorité.

CLI-08 Must En tant que développeur qui reçoit un lien de résultat, je veux le consulter sans créer de compte afin de juger la preuve par moi-même.

BO-10 Must
En tant qu'administrateur, je veux consulter le journal de toutes les actions d'administration afin de savoir qui a fait quoi et quand en cas d'incident.

Chaque story est reliée, dans une matrice de traçabilité, à ses écrans, à ses règles métier, aux risques qu'elle porte et aux parcours de bout en bout qui la vérifient.

04 · RISQUES

Le risque guide l'effort de test

Chaque risque est formulé comme un événement suivi de sa conséquence, puis coté : niveau = probabilité (1 à 3) × impact (1 à 3). Plus le niveau est élevé, plus le risque est testé tôt et en profondeur.



Répartition des 47 risques (34 produit, 13 projet) selon leur probabilité et leur impact.

30 risques de niveau 6 ou plus

Tout test lié à l'un de ces risques doit réussir pour que la version soit lancée.

Enseignement principal

Pour un produit qui vend de la confiance, les risques les plus graves portent sur **l'honnêteté du résultat**, pas sur les pannes.

Extrait du registre des risques

Risque	P	I	Niveau	Réponse de test
Le score est faux (défaut inventé ou grave manqué) : la confiance est détruite	3	3	9	Sites de référence aux défauts connus ; stabilité entre deux analyses
Le périmètre est trop lourd pour l'équipe : la date de lancement glisse	3	3	9	Estimation de l'effort avant de fixer la date ; Must d'abord
Un client voit les données d'un autre	2	3	6	Accès croisés entre deux organisations, par l'écran et par l'API
Une analyse démarre sans consentement	2	3	6	Tests par l'écran et par l'API ; preuve conservée
Le score est affiché sans dire ce qu'il ne couvre pas	2	3	6	Mention de périmètre vérifiée partout où le score apparaît
Le produit est lent sur réseau mobile : abandon	3	2	6	Mesure du premier affichage en 3G simulée

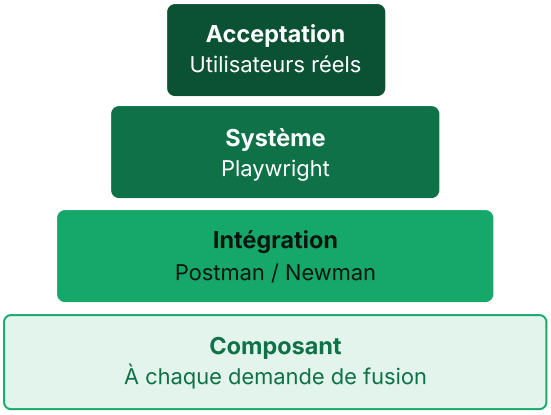
Deux familles de risques

Type de risque	Niveau 9	Niveau 6	Niveau 3 à 4	Total
Produit · ce qui peut mal fonctionner	1	21	12	**34**
Projet · ce qui peut empêcher de livrer	1	7	5	**13**

05 · APPROCHE

Comment le produit est testé

Beaucoup de tests rapides à la base, peu de tests lents au sommet.



Technique	Exemple d'application
Valeurs limites	Seuil de score testé à S−1, S et S+1 ; durées d'expiration
Partitions	Formats d'adresse : valide, sans protocole, invalide, non autorisée
Table de décision	Action recommandée selon le score et les défauts critiques
Transitions d'état	Cycle de vie d'une analyse, d'un compte, d'un lien partagé
Exploratoire	Sessions de 60 minutes sur les états d'échec

Contrôle d'accès testé là où il se joue

Une matrice croise chaque action avec chaque rôle. Chaque refus est testé par l'écran **et** par un appel direct à l'API ; un seul refus manquant est une anomalie bloquante.

Acceptation en trois volets

Le produit s'analyse lui-même ; des sites de référence aux défauts connus vérifient la justesse du score ; des utilisateurs réels font le parcours sans aide.

À chaque demande de fusion

Composant	API	Régression	Relecture	Auto-analyse
Tests unitaires	Postman / Newman	Suite Playwright	Validation du code	ProofQA se scanne

Une demande en échec est refusée ; un défaut critique détecté par l'auto-analyse bloque la mise en production.

06 · CRITÈRES

Une décision de lancement écrite à l'avance

Extrait des 19 critères de sortie

- 100 % des tests des Must exécutés
- 100 % des tests liés à un risque de niveau 6 ou plus réussis
- Aucune anomalie bloquante ; au plus deux majeures, avec un contournement accepté par écrit
- Sur les sites de référence : 100 % des défauts critiques détectés, aucun faux défaut critique
- 100 % des tests de contrôle d'accès réussis
- Premier affichage en moins de 3 s en 3G ; accessibilité Lighthouse ≥ 95

Indépendance du test. Le Product Owner cumulant la conduite du projet et le test, les critères de sortie sont écrits à l'avance et partagés avec toute l'équipe. Le jour de la décision, c'est le document qui tranche.

07 · ACCEPTATION

Des critères lisibles et automatisables

Les critères d'acceptation sont rédigés en Gherkin : lisibles par toute l'équipe et directement automatisables. Chaque scénario vérifie un seul comportement observable.

Un consentement obligatoire, vérifié jusqu'à l'API

```
Fonctionnalité : Consentement avant l'analyse

  Scénario : Bouton verrouillé tant que la case n'est pas cochée
    Étant donné que le visiteur a saisi l'adresse de son produit
    Quand l'écran de consentement s'affiche
    Alors la case n'est pas cochée
    Et le bouton "Lancer l'analyse" est verrouillé, avec la raison affichée

  Scénario : Contournement par l'API
    Quand une requête est envoyée directement à l'API sans consentement
    Alors l'API refuse la requête
    Et aucune analyse n'est créée en base de données
```

Une table de décision exprimée en Gherkin

```
Fonctionnalité : Action recommandée

  Plan du scénario : Selon le score et les défauts critiques
    Étant donné un seuil S
    Quand le score est <score> et qu'un défaut critique est <défaut>
    Alors l'action recommandée est "<action>"

    Exemples :
      | score | défaut  | action                      |
      | S - 1 | absent  | Parler à un expert          |
      | S     | absent  | Parler à un expert          |
      | S + 1 | absent  | Voir ce qui reste à vérifier |
      | S + 1 | présent | Parler à un expert          |
```

Ce que cette stratégie démontre

- **Le test commence avant le code :** la relecture croisée de la stratégie et des maquettes a corrigé dix-neuf écarts avant toute ligne de développement.
- **Le risque guide l'effort :** les risques les plus graves bloquent le lancement.
- **La sécurité est testée là où elle se joue :** par l'interface et par l'API.
- **La décision de lancement est objective :** elle repose sur des critères écrits à l'avance.

Ghislain TOUSSI KAMGA
QA Engineer · Test Lead · ISTQB Foundation v4
gtoussikamga@gmail.com
contact@proofqa.io

Version de présentation. Les valeurs de sécurité, les règles internes, les données de test et les procédures de la stratégie complète ne sont pas publiées. Aucun élément de calcul du score n'est publié.